

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Soundariya M	Reg. No.:	192524342
Section / Batch:	B.tech - AI&DS	Date:	11.08.2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

QUESTION1

CO2-AT3-Degugging

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

2/2 answered

Q1 Brute Force

50 marks

Debugging Pair Sum Code (Duplicate and Mirror Pair Detection)

The following brute force pair sum implementation may return duplicate or mirror index pairs for the same solution, increasing redundancy in output. Carefully analyze the nested loop structure and identify why repeated pair detection occurs. Debug the algorithm to ensure that each valid pair is checked only once without missing any solution. Optimize the implementation by reducing redundant comparisons while preserving correctness. Analyze the time complexity of the corrected solution. Rewrite the final optimized code with proper comments.

```
def pair_sum_all(arr, target):
    pairs = []
    for i in range(len(arr)):
        for j in range(len(arr)):
            if i != j and arr[i] + arr[j] == target:
                pairs.append((i, j))
    return pairs
```

Your Code

```
1 def pair_sum(arr, target):
2     pairs = []
3     for i in range(len(arr)):
4         for j in range(i + 1, len(arr)):
5             if arr[i] + arr[j] == target:
6                 pairs.append((i, j))
7     return pairs
8 n = int(input())
9 arr = list(map(int, input().split()))
10 target = int(input())
11
12 result = pair_sum(arr, target)
13
14 for pair in result:
15     print(pair[0], pair[1])
```

Submitted

Input

5
2 7 11 15 5
9

Output

0 1

QUESTION 2

CO2-AT3-Degugging

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Brute Force
50 marks

Q2
Brute Force
50 marks

2/2 answered

Q2 Brute Force

50 marks

Debugging Selection Sort Code (Minimum Index Not Reset Properly)

The following Selection Sort implementation fails because the minimum index is initialized only once outside the outer loop, causing comparisons in later passes to use an outdated reference. Carefully analyze why the algorithm loses track of the current unsorted portion. Apply systematic debugging to reset the minimum index correctly in every pass. Optimize the implementation by avoiding unnecessary swaps when the selected minimum is already in place. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    min_idx = 0 # ERROR
    for i in range(n):
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Your Code

```
1 def selection_sort(arr):
2     n = len(arr)
3     for i in range(n - 1):
4         min_idx = i
5         for j in range(i + 1, n):
6             if arr[j] < arr[min_idx]:
7                 min_idx = j
8         if min_idx != i:
9             arr[i], arr[min_idx] = arr[min_idx], arr[i]
10    return arr
11 n = int(input())
12 arr = list(map(int, input().split()))
13 result = selection_sort(arr)
14 print(*result)
```

Submitted

Input

5
64 25 12 22 11

Output

11 12 22 25 64

Code of Conduct

I Soundariya M(192524342)(*Name / Reg No*) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Soundariya M**

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____